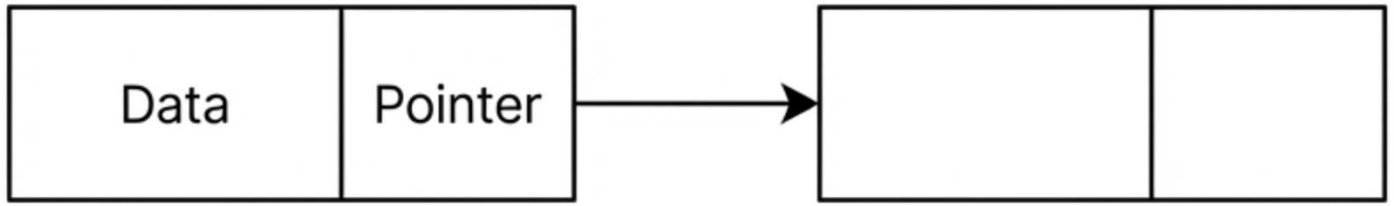
Universidad del Rosario | Escuela de Ciencias e Ingeniería | MACC

Listas Enlazadas

Implementación y optimización

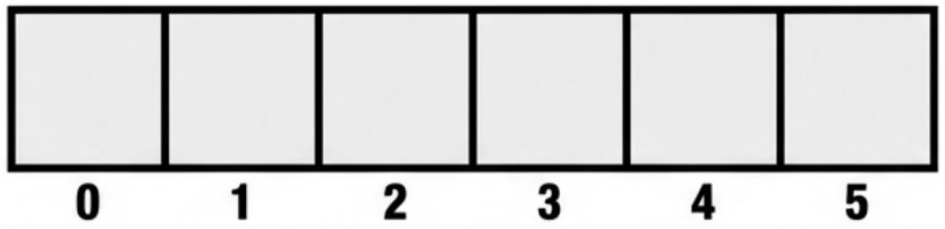


Objetivos

- Diferenciar entre arreglos y listas enlazadas a nivel de gestión de memoria (Stack vs. Heap).
- Construir la estructura atómica Node utilizando punteros y Templates en C++.
- Implementar la clase SinglyLL con operaciones fundamentales de inserción (pushFront, pushBack, insertAt).
- Gestionar la memoria dinámica correctamente, implementando destructores y operaciones de eliminación (pop, removeAt, clear) para evitar *Memory Leaks*.

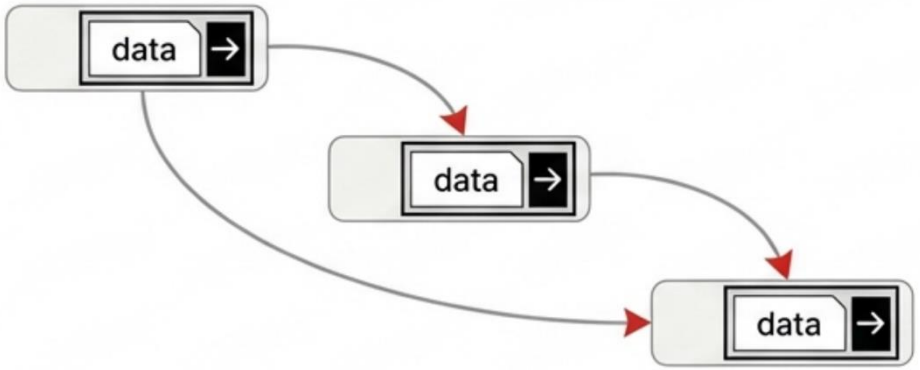
Arreglos vs Listas

Arreglos (Arrays)



- Memoria Contigua: Bloques adyacentes.
- Acceso O(1) (Rápido).
- Tamaño fijo (Costoso de redimensionar).

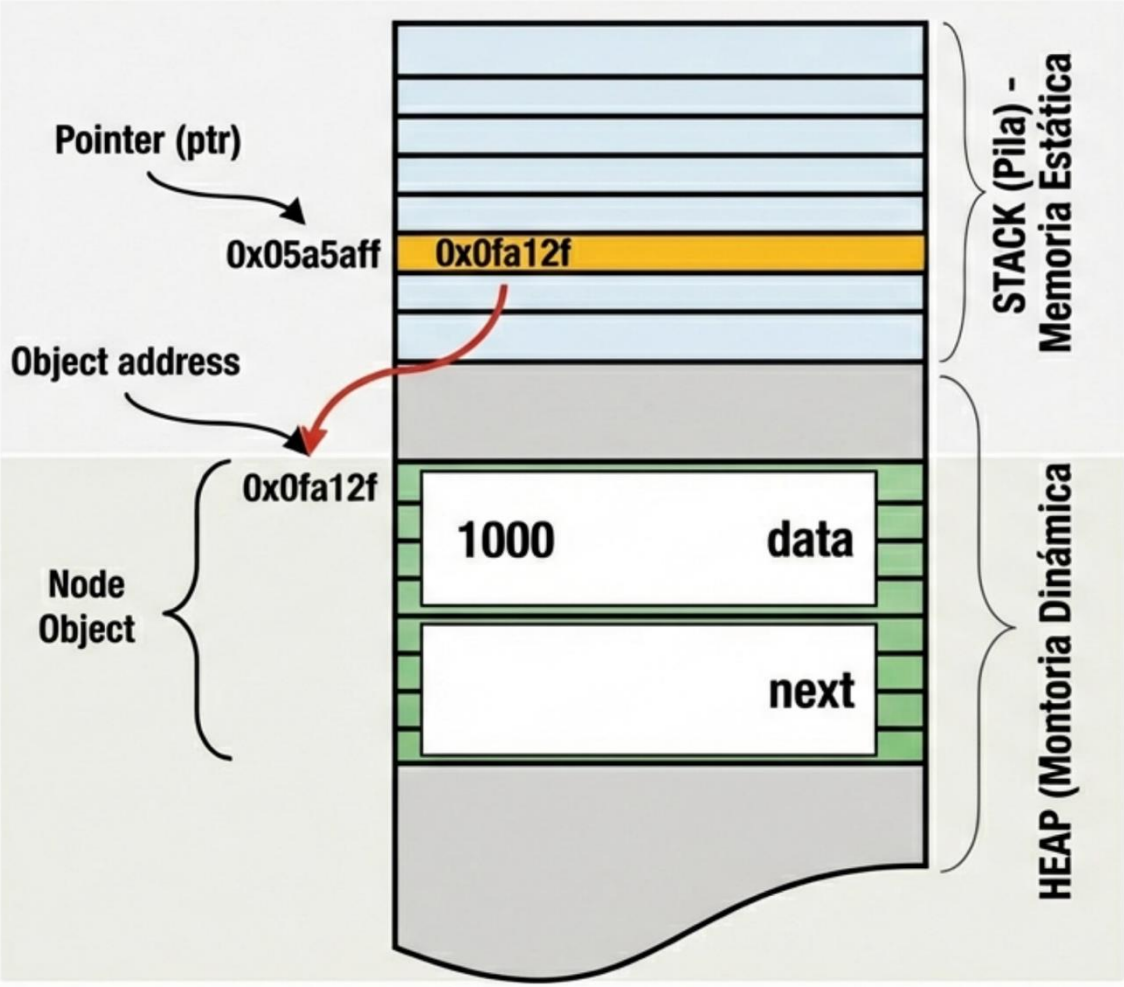
Listas Enlazadas (Linked Lists)



- Memoria Dispersa: Nodos unidos por punteros.
- Acceso O(n) (Secuencial).
- Crecimiento dinámico y eficiente.

Insight Clave: A diferencia de los arreglos, las listas no requieren un bloque continuo de memoria, permitiendo una asignación eficiente en sistemas fragmentados.

Gestión de Memoria: Stack vs. Heap

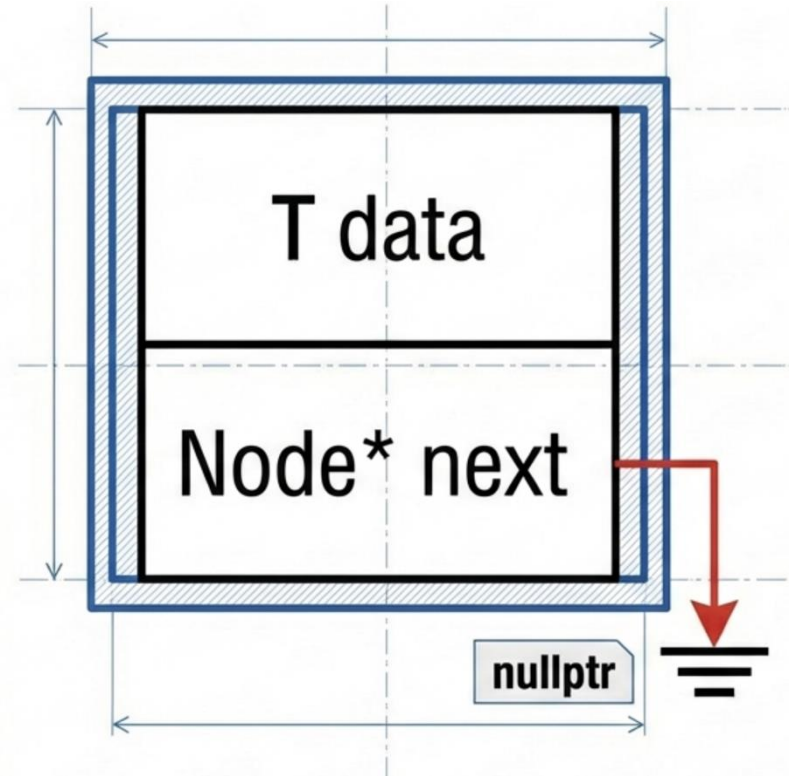



```
Node* ptr = new Node;
// 'ptr' vive en Stack
// apunta a memoria en Heap
```

Stack: Almacena la dirección de memoria (rápido).

Heap: Almacena el objeto real (flexible, manual).

Nodo: struct Node




```
template <typename T>
struct Node {
  T data;        // El valor almacenado
  Node* next;    // El enlace al siguiente

  // Constructor
  Node(const T& item) : data(item), next(nullptr) {}
};
```

Templates (**<T>**): Permite almacenar cualquier tipo de dato.

nullptr: Inicialización segura (C++17) para evitar punteros colgantes.

La clase **SinglyLL**

La interfaz que administra el ciclo de vida de los nodos.

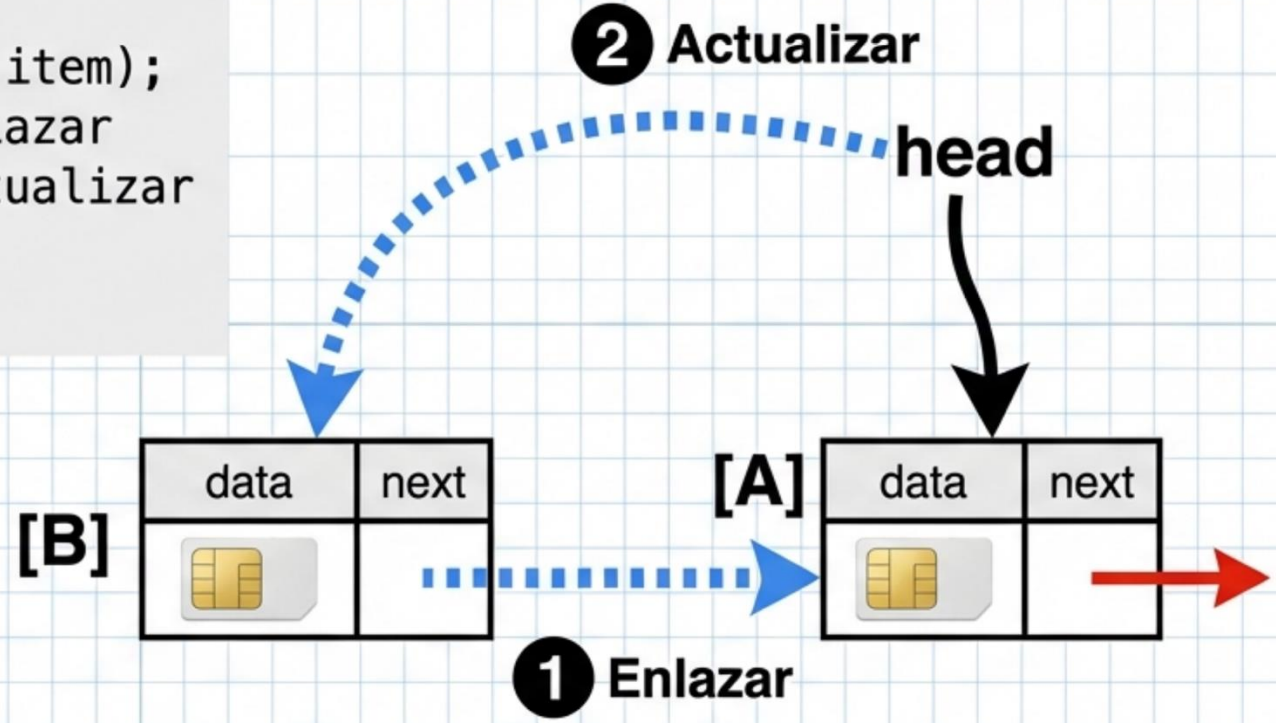
```
template <typename T>
class SinglyLL {
private:
    Node<T>* head;  // El punto de entrada único
    size_t size;    // Rastreo del número de elementos
public:
    SinglyLL() : head(nullptr), size(0) {}

    // Operaciones Principales
    void pushFront(const T item);
    void pushBack(const T item);
    void insertAt(size_t index, const T item);

    // ... destructores y métodos de acceso
};
```

¡Crucial! Si perdemos 'head', perdemos el acceso a toda la lista (**Memory Leak**).

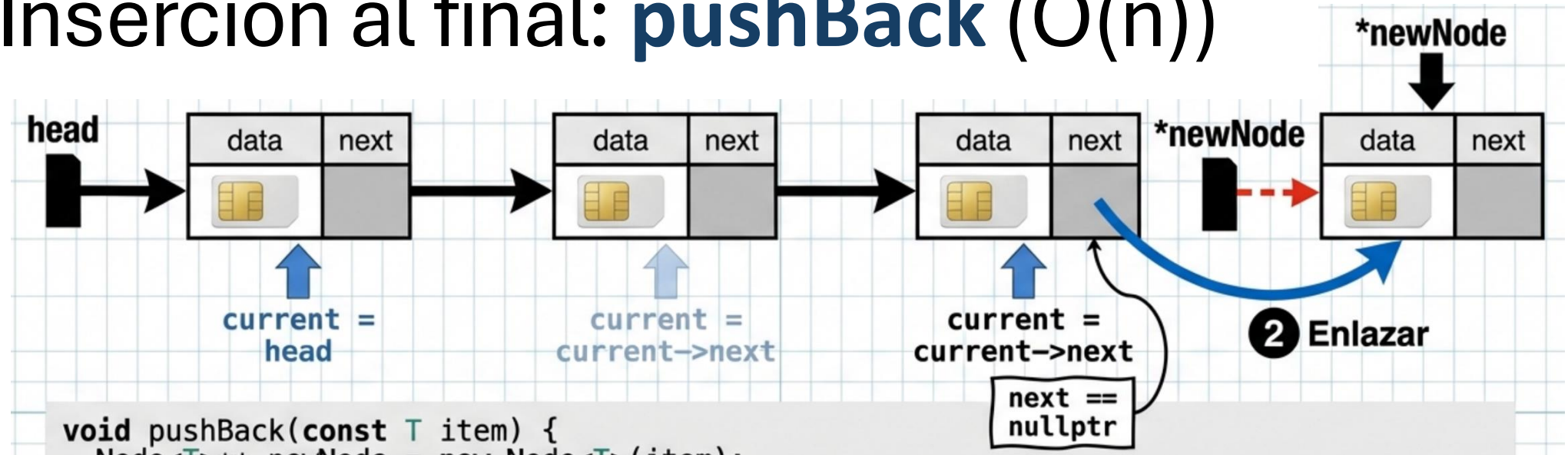
Inserción al Inicio: **pushFront** (O(1))

```
void pushFront(const T item) {
  Node<T>* newNode = new Node<T>(item);
  newNode->next = head; // 1. Enlazar
  head = newNode;       // 2. Actualizar
  size++;
}
```

Rendimiento constante. No requiere recorrer la lista.



Inserción al final: **pushBack** (O(n))



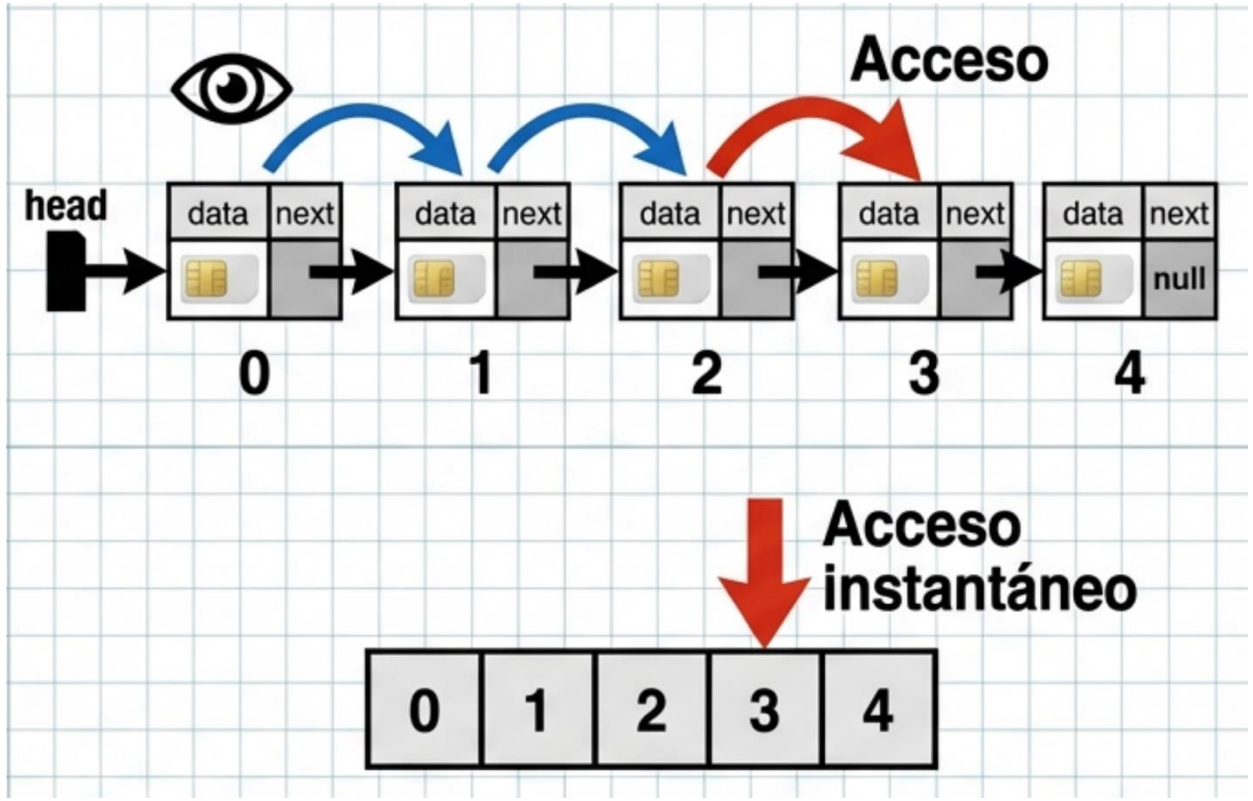

```
void pushBack(const T item) {
  Node<T>** newNode = new Node<T>(item);
  if (!head) {
      head = newNode;       // Caso 1: Lista vacía
  } else {
      Node<T>* current = head;
      while (current->next) {     // Recorrer
          current = current->next;
      }
      current->next = newNode;    // Enlazar
  }
  size++;
}
```

Rendimiento lineal (O(n)) debido al recorrido completo de la lista.

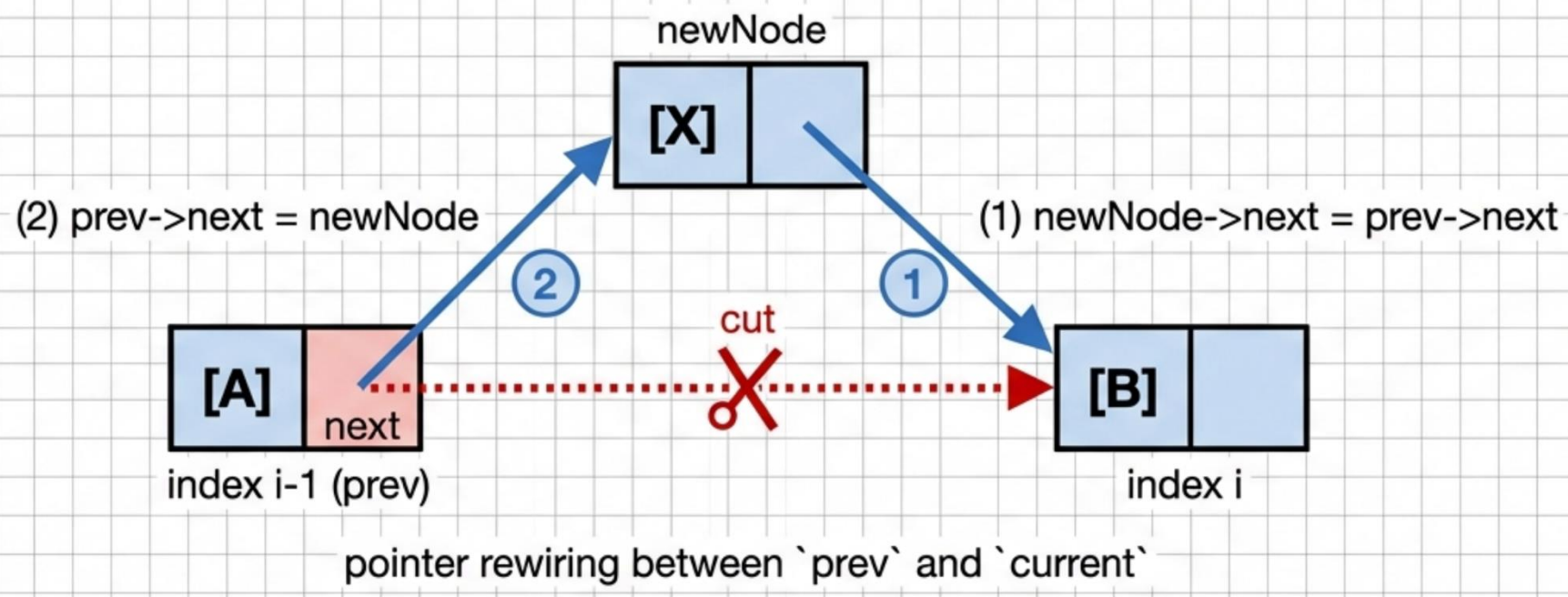
Acceso a Elementos: Navegación Lineal

```
T& get(size_t index) {
  Node<T>* current = head;
  for (size_t i = 0; i < index; ++i) {
    current = current->next; // "Caminar"
  }
  return current->data;
}

T& operator[](size_t i) { return get(i); }
```

Costo: k pasos para llegar al índice k. (O(n)).

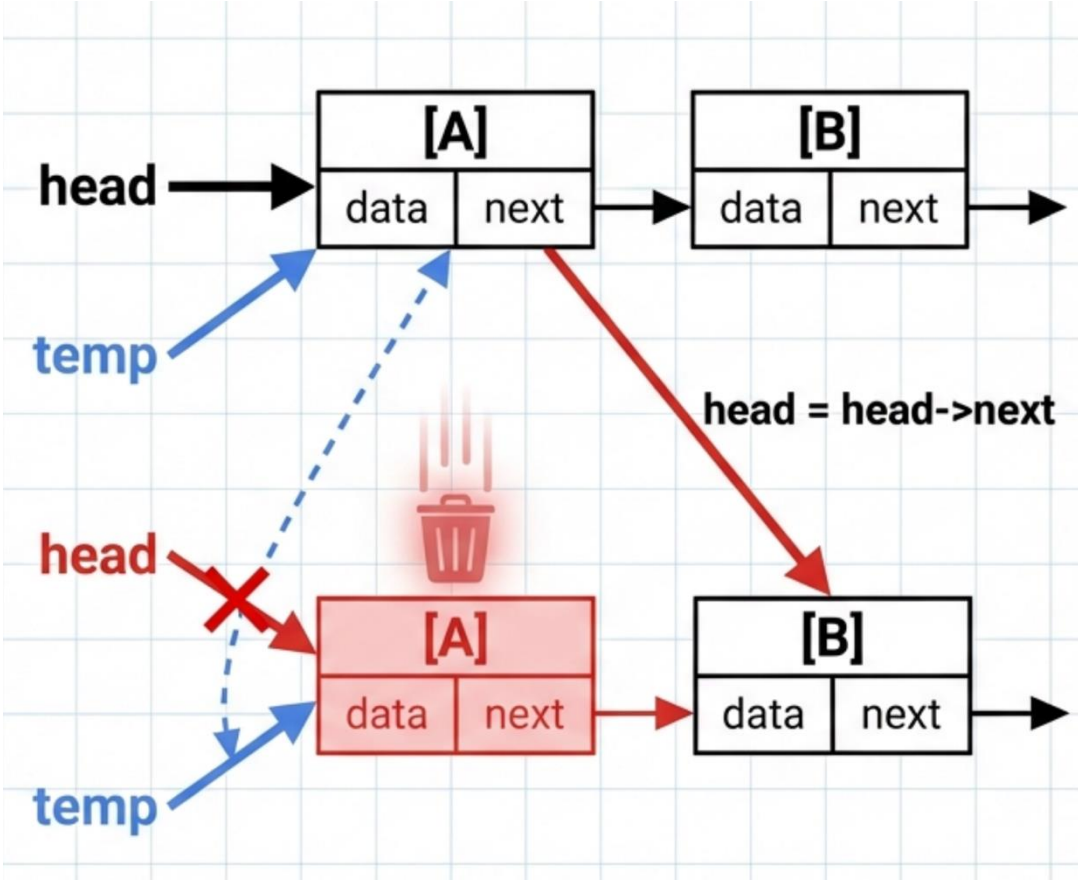


Precisión Quirúrgica: **insertAt**



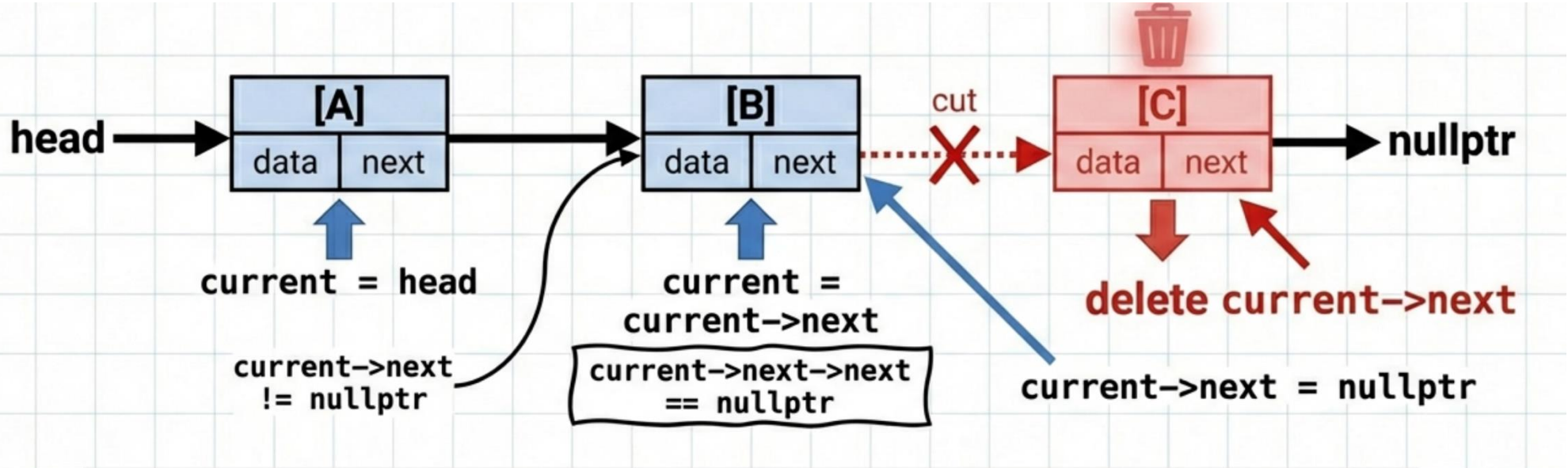

```
// ... travesía hasta 'prev' ...
newNode->next = prev->next; // Enlazar nuevo al siguiente
prev->next = newNode;       // Enlazar previo al nuevo
```

Eliminación al Inicio: **popFront**



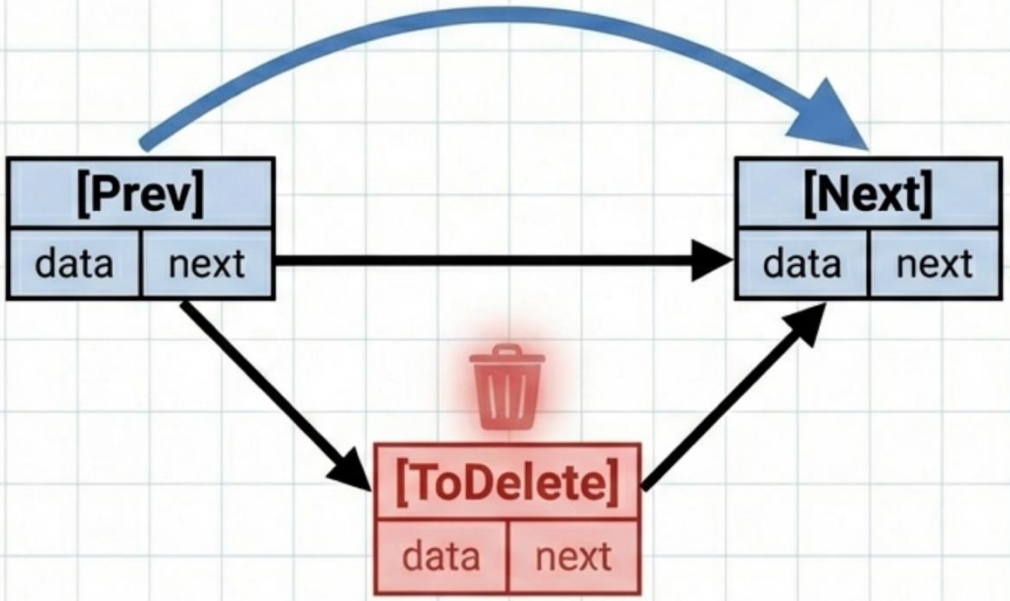

```
T popFront() {
  Node<T>* temp = head;
  T data = head->data;

  head = head->next; // Avanzar head
  delete temp;       // ¡Liberar memoria!

  size--;
  return data;
}
```

Eliminación al Final: **popBack**




```
// Travesía hasta el penúltimo nodo
while (current->next->next) {
    current = current->next;
}
delete current->next;       // Eliminar el último
current->next = nullptr;    // Cortar el enlace
```

Extracción Selectiva: **removeAt**




```
  Node<T>* toDelete = current->next;
  current->next = toDelete->next;
  current->next = toDelete->next;
                  // "Puenteo"
  delete toDelete; // Limpieza
}
```

Visualizando los Datos: **print()**

```
void print() const {
  Node<T>* current = head;
  while (current != nullptr) {
    std::cout << current->data << " -> ";
    current = current->next;
  }
  std::cout << "nullptr";
}
```

```
> 10 -> 20 -> 30 -> 42 -> nullptr
```

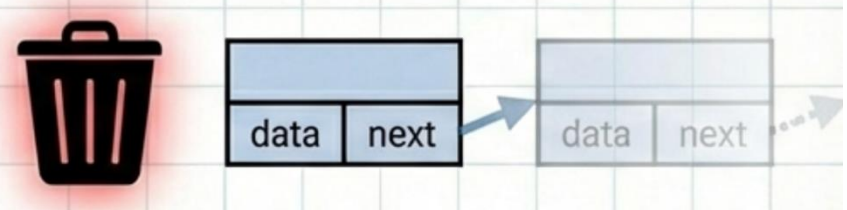
El bucle **while(current != nullptr)** es el patrón estándar de recorrido.

Ciclo de Vida y Limpieza: El Destructor

RAII: Resource Acquisition Is Initialization.

En C++, la memoria no se limpia sola. Debemos evitar Memory Leaks.

```
void clear() {
  while (head != nullptr) {
    popFront(); // Reutilizar lógica existente
  }
}

~SinglyLL() { clear(); }
```



Resumen

1. Arquitectura de Memoria:
Hemos trascendido la limitación de la memoria contigua de los Arreglos (std::vector).

- **Arreglos:** Bloques rígidos y adyacentes. Rápidos de leer, costosos de redimensionar.
- **Listas:** Memoria dispersa en el *Heap*. Nodos conectados lógicamente mediante punteros, permitiendo un crecimiento dinámico sin reasignación masiva.

2. El Átomo de la Estructura: struct Node<T>
Construimos una unidad fundamental genérica utilizando **Templates**.

- **Composición:** Una carga útil (data) y un puntero de navegación (next).
- **Optimización Moderna:** Implementamos **Semántica de Movimiento (std::move)** en el constructor, permitiendo transferir recursos de objetos pesados en O(1) en lugar de copiarlos bit a bit.

3. Algoritmos de Manipulación (Punteros)
Aprendimos que la eficiencia radica en el "recableado" inteligente de punteros, no en el desplazamiento de datos.

- **Inserción al Inicio (pushFront):** Operación estrella de la lista. Costo constante O(1).
- **Inserción al Final (pushBack):** Costo lineal O(n) (en nuestra implementación simple sin tail), ya que requiere recorrer toda la cadena.
- **Gestión de Memoria (RAII):** Implementamos un **Destructor** robusto que utiliza clear() y popFront() para liberar secuencialmente cada nodo del *Heap*, evitando fugas de memoria (*Memory Leaks*).

No existe la estructura de datos perfecta, solo la adecuada para el problema. Utilizaremos **Listas Enlazadas** cuando la prioridad sea la inserción y eliminación constante de elementos (especialmente al inicio) y no necesitemos acceso aleatorio veloz. Para todo lo demás, la memoria contigua del **Vector** sigue siendo el estándar de la industria.